

Manual del Curso

Lenguaje de trabajo: **Python 3**

Nivel: Universitario – Introductorio a intermedio

Cátedra de Algoritmos y Estructuras de Datos

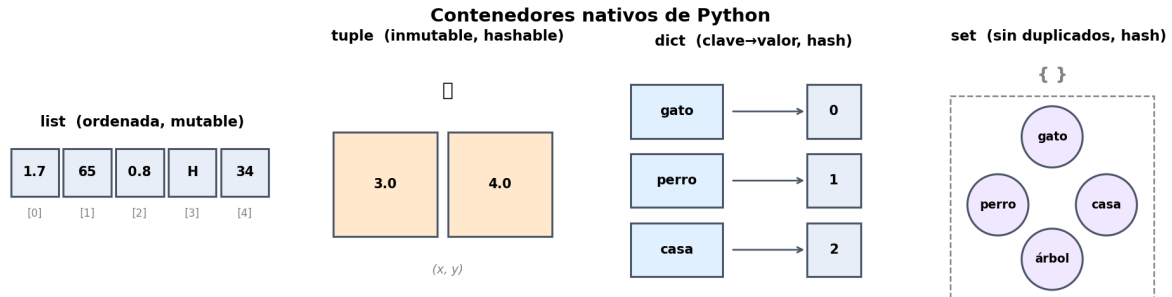
Índice de contenidos

#	Tema	Pág.
1	Contenedores nativos (list, tuple, dict, set)	3
2	Comprensión de listas	6
3	Tipos de Datos Abstractos (TDA)	7
4	Pilas y Colas	9
5	Colas de prioridad	11
6	Heap	12
7	Árboles y árboles binarios	14
8	Recorridos: preorden, inorden, postorden	16
9	Grafos y sus recorridos: BFS y DFS	20
10	Lista y matriz de adyacencia	24
11	Complejidad temporal y espacial	26

■ **Nota IA:** Este manual utiliza **diagramas paso a paso** (frames) que simulan la animación de cada recorrido. Al leer las secuencias de imágenes de izquierda a derecha y de arriba hacia abajo, se puede reconstruir mentalmente la ejecución del algoritmo, como si se estuviera viendo un video.

1. Contenedores nativos de Python

Los **contenedores** son objetos que agrupan otros objetos. Python ofrece cuatro contenedores nativos fundamentales: **listas**, **tuplas**, **diccionarios** y **conjuntos**. Elegir bien entre ellos impacta directamente en el rendimiento de un algoritmo de IA, donde muchas veces trabajamos con millones de elementos.



Los cuatro contenedores nativos con su forma de acceso característica.

1.1 Listas (list)

Una lista es una secuencia **ordenada** y **mutable**. Internamente CPython la implementa como un arreglo dinámico de punteros: reserva más memoria de la necesaria y la duplica cuando se llena, lo que le permite operaciones *append* en $O(1)$ amortizado.

```
features = [1.7, 65.2, 0.87, 'hombre', 34]
features.append(0.95)      # 0(1) amortizado
features[0] = 1.75         # 0(1) acceso por índice
features.insert(0, 'id')   # 0(n) hay que correr todos
del features[2]            # 0(n)
```

■ **Nota IA:** Cuando entrenamos un modelo, muchas veces almacenamos temporalmente lotes de datos en listas antes de convertirlos a `numpy.ndarray` o `torch.Tensor`. Las listas son cómodas pero **lentas** para operaciones numéricas: no vectorizan. Un producto punto sobre una lista de 1M elementos con un `for` tarda segundos; con NumPy, milisegundos.

1.2 Tuplas (tuple)

Secuencia **ordenada** e **inmutable**. Por ser inmutable, Python la usa como **clave de diccionarios** o elemento de conjuntos (es *hashable*).

```
punto = (3.0, 4.0)          # coordenada en R²
estado = ((1,2,3), (4,0,5), (6,7,8))
visitados = set()
visitados.add(estado)       # 0(1) promedio
```

■ **Nota IA:** Las tuplas son perfectas para representar **estados** en algoritmos de búsqueda (A*, minimax). Como son hashables, podemos guardarlas en un set de estados visitados. En un 8-puzzle cada tablero es una tupla de tuplas.

1.3 Diccionarios (dict)

Estructura clave-valor implementada como **tabla hash**. Desde Python 3.7 mantiene el orden de inserción. Operaciones promedio $O(1)$ para inserción, búsqueda y borrado.

```
vocab = {'gato': 0, 'perro': 1, 'casa': 2}
vocab['árbol'] = 3      # inserción 0(1) promedio
idx = vocab['gato']     # búsqueda 0(1) promedio
```

```
from collections import Counter
palabras = 'el gato come el gato duerme'.split()
Counter(palabras) # {'el':2, 'gato':2, ...}
```

■ **Nota IA:** Los diccionarios son omnipresentes en IA: vocabularios en NLP (word2idx), **memoización** en programación dinámica y Viterbi, tablas de transposición en juegos, conteo de frecuencias para Naive Bayes.

1.4 Conjuntos (set)

Colección **no ordenada**, **sin duplicados** y mutable. Igual que el dict, se basa en tabla hash. Operaciones fundamentales O(1) promedio.

```
a = {'gato', 'mamífero', 'doméstico'}
b = {'perro', 'mamífero', 'doméstico'}
a & b # intersección
a | b # unión
a - b # diferencia

def jaccard(a, b):
    return len(a & b) / len(a | b) if a|b else 0
```

■ **Nota IA:** Los conjuntos son ideales para **estados visitados** en BFS/DFS (evitar ciclos) y para la **similitud de Jaccard**, base de sistemas de recomendación y detección de plagios.

Tabla comparativa

Contenedor	Ordenado	Mutable	Duplicados	Indexable	Hashable
list	Sí	Sí	Sí	Sí	No
tuple	Sí	No	Sí	Sí	Sí*
dict	Sí (ins.)	Sí	Claves: no	Por clave	No
set	No	Sí	No	No	No

** si sus elementos son hashables*

2. Comprensión de listas

Sintaxis declarativa para construir listas a partir de iterables, más rápida y legible que un bucle for tradicional. Su forma general es:

```
[expresión for elemento in iterable if condición]
```

Ejemplos progresivos:

```
# Cuadrados del 0 al 9
cuadrados = [x**2 for x in range(10)]

# Filtrado: solo pares
pares = [x for x in range(20) if x % 2 == 0]

# Doble bucle: producto cartesiano
puntos = [(x, y) for x in range(3) for y in range(3)]

# Comprensión anidada: matriz identidad 3x3
I = [[1 if i==j else 0 for j in range(3)] for i in range(3)]
```

Aplicaciones IA

```
# Normalización min-max
datos = [12, 45, 78, 33, 90, 5]
mn, mx = min(datos), max(datos)
norm = [(x - mn) / (mx - mn) for x in datos]

# Tokenización simple
frase = 'El Perro Corre RÁPIDO'
tokens = [w.lower() for w in frase.split() if len(w) > 2]

# One-hot encoding manual
clases = ['gato', 'perro', 'pájaro']
etiqueta = 'perro'
one_hot = [1 if c == etiqueta else 0 for c in clases]
```

Variantes hermanas

```
# Comprensión de diccionario
word2idx = {w: i for i, w in enumerate(['gato', 'perro', 'casa'])}

# Comprensión de conjunto
vocab = {w.lower() for w in 'El el ELLO'.split()}

# Expresión generadora (lazy, ahorra memoria)
suma = sum(x**2 for x in range(10**7))
```

■ **Nota IA:** No abusen del anidamiento. Si necesitan tres for y dos if, un bucle tradicional será más legible. La comprensión debe **aclarar**, no ocultar.

3. Tipos de Datos Abstractos (TDA)

Un **TDA** es una especificación matemática de un conjunto de datos y las operaciones válidas sobre ellos, **independiente de la implementación**. Define el *qué* (comportamiento) y oculta el *cómo* (representación interna). Este principio, llamado **encapsulamiento**, es la base de la ingeniería de software.

Componentes de un TDA

- **Dominio:** conjunto de valores válidos.
- **Operaciones:** firmas (nombres, parámetros, retornos).
- **Axiomas:** reglas de comportamiento (ej. `pop(push(P, x)) == x`).

Ejemplo: TDA Pila especificado formalmente

```
TDA Pila <T>::
    crear()          → Pila
    apilar(P, x)     → Pila
    desapilar(P)     → T
    tope(P)          → T
    vacia?(P)        → bool

Axiomas:
    vacia?(crear())      = True
    vacia?(apilar(P, x)) = False
    tope(apilar(P, x))   = x
    desapilar(apilar(P, x)) = P
```

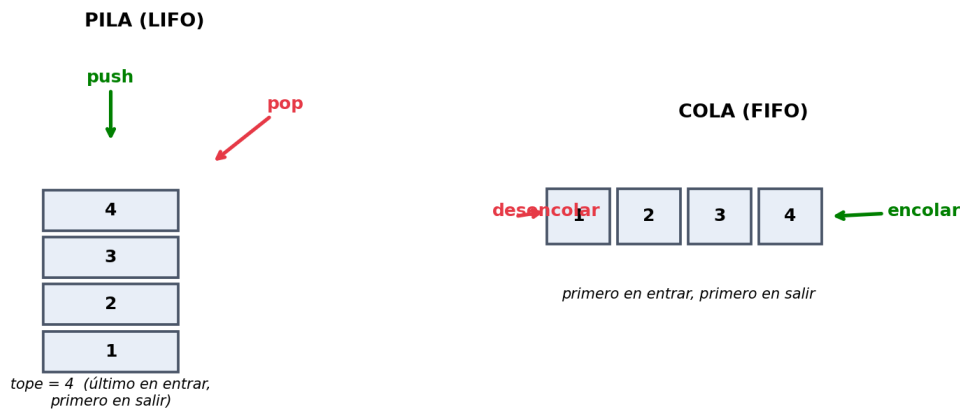
Fíjense que no se dice nada sobre arreglos, listas enlazadas ni punteros. Esa libertad de implementación es la esencia del TDA.

Implementación en Python

```
class Pila:
    def __init__(self):
        self._datos = [] # detalle oculto
    def apilar(self, x):
        self._datos.append(x)
    def desapilar(self):
        if not self._datos:
            raise IndexError('pila vacía')
        return self._datos.pop()
    def tope(self):
        return self._datos[-1]
    def vacia(self):
        return len(self._datos) == 0
    def __len__(self):
        return len(self._datos)
```

■ **Nota IA:** PyTorch define TDAs como `Dataset` y `DataLoader`: uno implementa `__len__` y `__getitem__`, y el framework maneja el resto (batching, shuffling, paralelismo). Esa es la potencia del pensamiento abstracto.

4. Pilas y Colas



Pila LIFO (izq): último en entrar, primero en salir. Cola FIFO (der): primero en entrar, primero en salir.

4.1 Pila (Stack) — LIFO

Last In, First Out. Como una pila de platos.

```
pila = []
pila.append(1)    # push   → O(1)
pila.append(2)
x = pila.pop()    # pop    → O(1) ← devuelve 2
```

Usar `list` con `append/pop` es óptimo porque ambas operaciones son $O(1)$ amortizado sobre el final.

Aplicaciones en IA:

- **DFS iterativo** (recorrido en profundidad de grafos y árboles de decisión).
- **Backtracking** en CSP como Sudoku o coloreo de grafos.
- **Recursión**: la propia pila de llamadas de Python es un stack.
- **Evaluación de expresiones** en compiladores.

4.2 Cola (Queue) — FIFO

First In, First Out. Como una fila en el banco.

```
from collections import deque
cola = deque()
cola.append(1)    # encolar   → O(1)
cola.append(2)
x = cola.popleft() # desencolar → O(1)
```

■ **Nota IA: ¡Nunca usen `list.pop(0)` para una cola!** Es $O(n)$ porque debe correr todos los elementos. La `deque` está implementada como lista doblemente enlazada de bloques, permitiendo ambos extremos en $O(1)$.

Aplicaciones en IA:

- **BFS** (búsqueda en anchura), central en pathfinding y redes sociales.
- **Replay buffer** en Reinforcement Learning (DQN).
- **Pipelines** productor-consumidor.

```
class ReplayBuffer:
    def __init__(self, capacidad=10000):
```

```
        self.buffer = deque(maxlen=capacidad)
def agregar(self, transicion):
    self.buffer.append(transicion)
def muestrear(self, k):
    import random
    return random.sample(self.buffer, k)
```

5. Colas de Prioridad

Una cola de prioridad es un TDA donde cada elemento tiene una prioridad, y la extracción siempre devuelve el elemento de **mayor** (o menor) prioridad, no el más antiguo. La FIFO se rompe: manda la prioridad.

```
Insertar: (3, 'tarea A'), (1, 'tarea B'), (5, 'tarea C')
Extraer:  (1, 'tarea B')  ← menor valor = mayor prioridad
Extraer:  (3, 'tarea A')
Extraer:  (5, 'tarea C')
```

Operaciones típicas

- `insertar(x, p)`: agrega x con prioridad p — $O(\log n)$
- `extraer_min()`: retira y devuelve el mejor — $O(\log n)$
- `ver_min()`: consulta sin retirar — $O(1)$

¿Por qué es tan importante en IA?

Porque prácticamente todos los algoritmos de **búsqueda informada** la usan:

- **Dijkstra**: extraer el nodo con menor distancia.
- **A***: extraer el nodo con menor $f(n) = g(n) + h(n)$.
- **Best-First Search**: extraer el nodo más prometedor.
- **Algoritmo de Huffman**: extraer los dos árboles menos frecuentes.
- **Beam Search** en generación de texto: mantener los top-k candidatos.

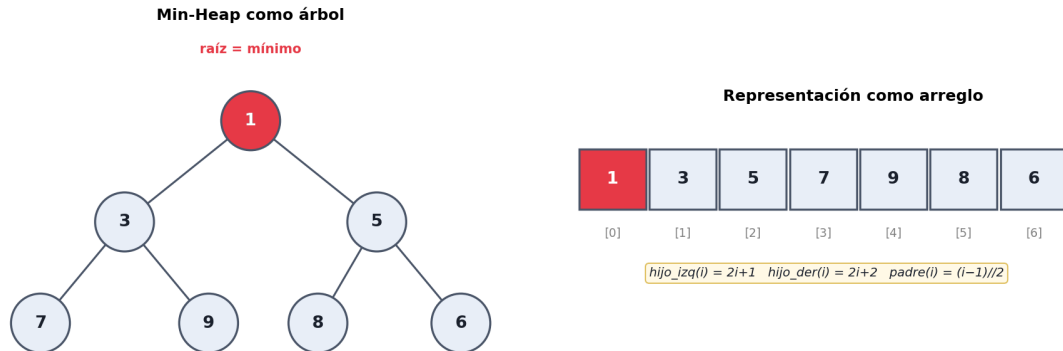
La forma más eficiente de implementarla es con un **Heap**, que veremos ahora.

6. Heap

Un **heap** es un **árbol binario casi completo** que satisface la propiedad de heap:

- **Min-heap**: cada nodo $\leq$ que sus hijos $\rightarrow$ la raíz es el mínimo.
- **Max-heap**: cada nodo $\geq$ que sus hijos $\rightarrow$ la raíz es el máximo.

Nota clave: **no está totalmente ordenado**, solo respeta la relación padre-hijo. Los hermanos pueden estar en cualquier orden.



Un min-heap visto como árbol y como arreglo. Los índices siguen aritmética simple.

Operaciones y complejidad

Operación	Complejidad
Ver mínimo (peek)	$O(1)$
Insertar (push) — sift-up	$O(\log n)$
Extraer mínimo (pop) — sift-down	$O(\log n)$
Construir heap desde lista (heapify)	$O(n)$

heapq en Python

```
import heapq

h = []
heapq.heappush(h, (5, 'C'))
heapq.heappush(h, (1, 'B'))
heapq.heappush(h, (3, 'A'))

heapq.heappop(h)  # (1, 'B')
heapq.heappop(h)  # (3, 'A')

# Construcción en  $O(n)$ 
datos = [5, 3, 8, 1, 9, 2]
heapq.heapify(datos)

# Para max-heap se invierte el signo:
# heapq.heappush(h, (-prioridad, item))
```

Ejemplo aplicado: A* en un grid

```
import heapq

def a_star(inicio, meta, vecinos, heuristica):
```

```

frontera = [(0, inicio)]      # (f_score, nodo)
g_score = {inicio: 0}
padre = {inicio: None}

while frontera:
    _, actual = heapq.heappop(frontera)
    if actual == meta:
        camino = []
        while actual is not None:
            camino.append(actual)
            actual = padre[actual]
        return camino[::-1]

    for vecino, costo in vecinos(actual):
        g = g_score[actual] + costo
        if g < g_score.get(vecino, float('inf')):
            g_score[vecino] = g
            f = g + heuristica(vecino, meta)
            padre[vecino] = actual
            heapq.heappush(frontera, (f, vecino))

return None

```

■ **Nota IA:** Sin heap, cada iteración de A* costaría $O(n)$ para encontrar el mínimo, degradando el algoritmo a $O(n^2)$. Con heap: **$O((V+E) \log V)$** .

7. Árboles y Árboles Binarios

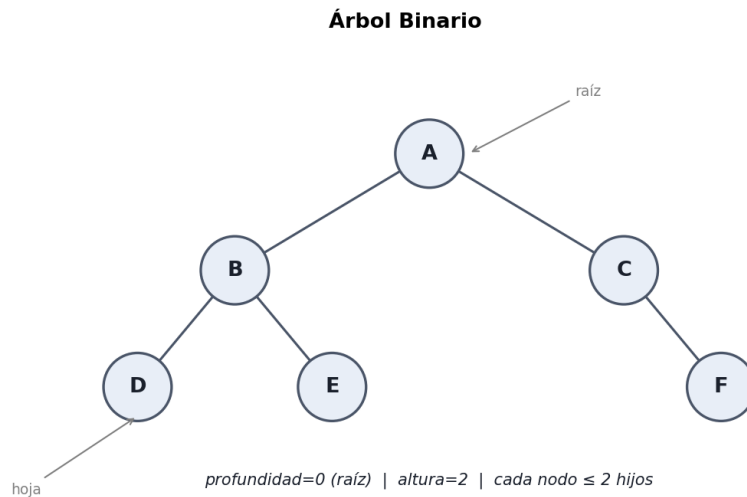
Un **árbol** es un grafo conexo acíclico con un nodo distinguido llamado **raíz**. Cada nodo tiene 0 o más **hijos** y exactamente un **padre** (salvo la raíz).

Terminología esencial

- **Raíz**: nodo sin padre.
- **Hoja**: nodo sin hijos.
- **Profundidad** de un nodo: distancia desde la raíz.
- **Altura** del árbol: profundidad máxima.
- **Grado**: cantidad de hijos de un nodo.

Árbol Binario

Cada nodo tiene **como máximo dos hijos** (izquierdo y derecho).



Árbol binario con 6 nodos. Raíz A, hojas D, E y F.

Tipos importantes

- **Binario completo**: todos los niveles llenos.
- **Binario casi completo**: como el heap.
- **BST** (Árbol Binario de Búsqueda): $izq < nodo < der$. Búsqueda promedio $O(\log n)$, peor $O(n)$.
- **Balanceados** (AVL, Rojo-Negro): mantienen altura $O(\log n)$ garantizada.

Implementación en Python

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izq = None
        self.der = None

class BST:
    def __init__(self):
        self.raiz = None
    def insertar(self, v):
        self.raiz = self._insertar(self.raiz, v)
```

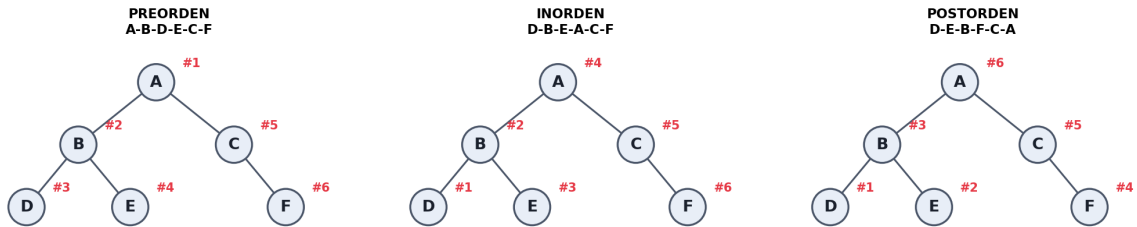
```
def _insertar(self, nodo, v):
    if nodo is None: return Nodo(v)
    if v < nodo.valor:
        nodo.izq = self._insertar(nodo.izq, v)
    else:
        nodo.der = self._insertar(nodo.der, v)
    return nodo
def buscar(self, v):
    return self._buscar(self.raiz, v)
def _buscar(self, nodo, v):
    if nodo is None or nodo.valor == v: return nodo
    if v < nodo.valor: return self._buscar(nodo.izq, v)
    return self._buscar(nodo.der, v)
```

■ **Nota IA:** Los árboles son la columna vertebral de muchos algoritmos: **árboles de decisión** (ID3, C4.5, CART), **Random Forest**, **XGBoost/LightGBM** (state-of-the-art en datos tabulares), **minimax** en juegos, **MCTS** (Monte Carlo Tree Search, motor de AlphaGo) y **KD-Tree** para búsqueda de vecinos en KNN.

8. Recorridos de árboles

Son los tres **recorridos en profundidad (DFS)** clásicos de un árbol binario. Se diferencian en **cuándo se visita la raíz** respecto a sus subárboles. En las siguientes páginas se muestra cada recorrido paso a paso, como una animación: el **nodo naranja** es el que se visita en ese paso, y los **verdes** son los ya visitados.

Vista comparativa



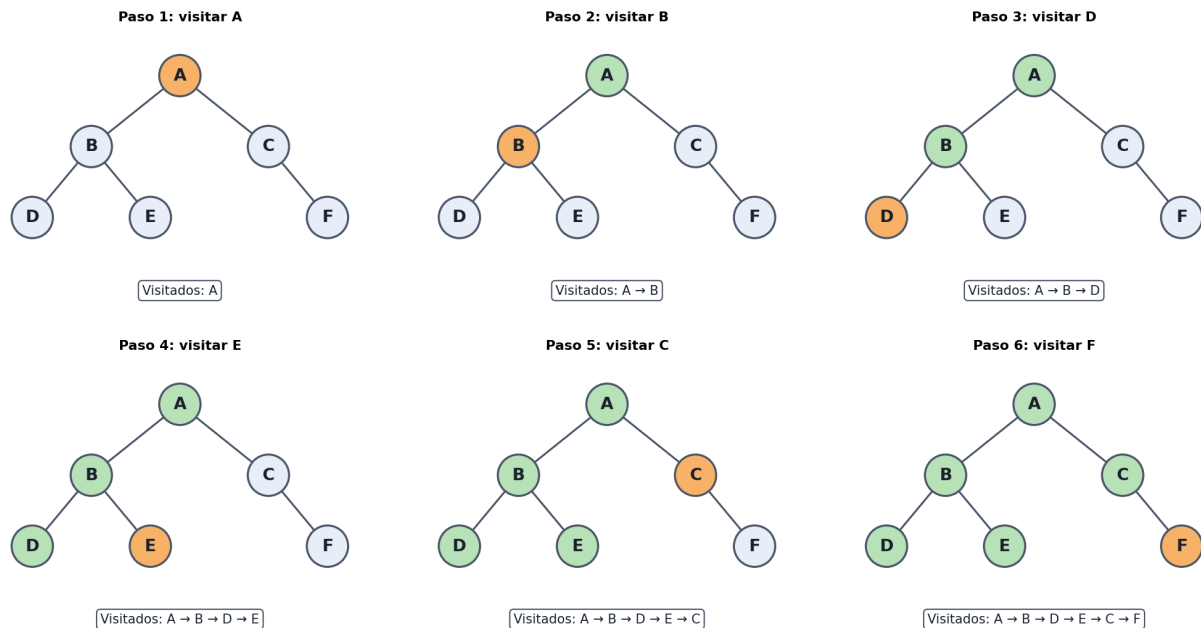
El orden de visita (#1, #2, ...) en cada estrategia sobre el mismo árbol.

8.1 Preorden: Raíz → Izquierda → Derecha

Resultado: A, B, D, E, C, F

Útil para **copiar** o **serializar** un árbol: al reconstruir después, procesamos primero cada padre.

Recorrido PREORDEN (Raíz → Izq → Der)



Animación paso a paso del recorrido preorden.

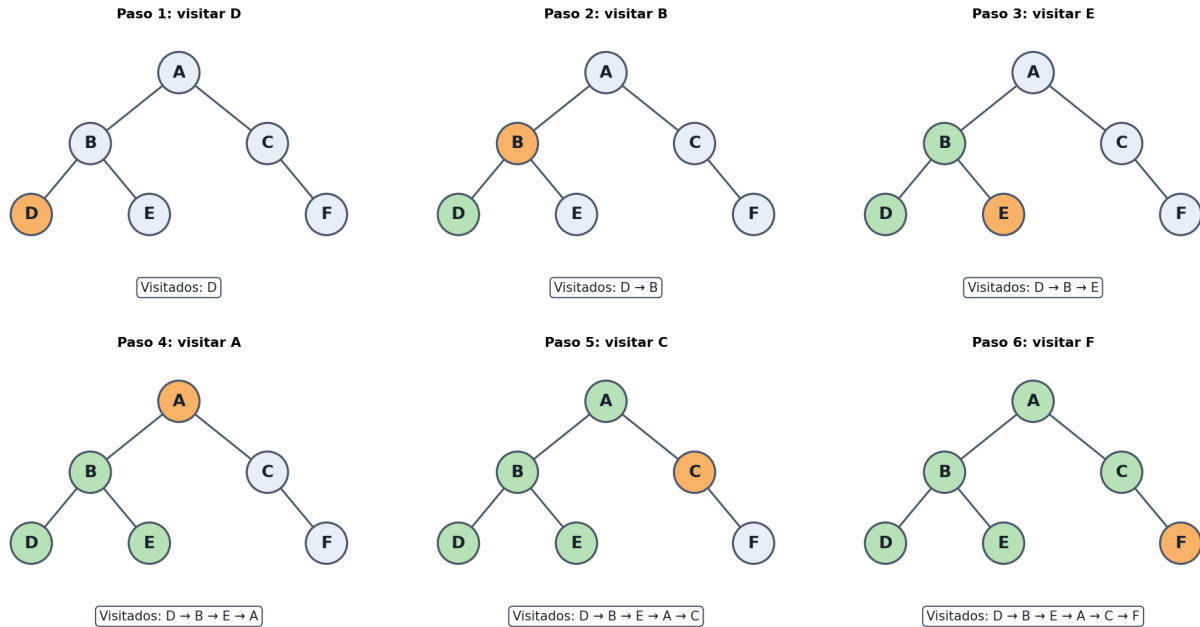
```
def preorden(nodo, visitar):  
    if nodo:  
        visitar(nodo.valor)      # 1º: raíz  
        preorden(nodo.izq, visitar) # 2º: izquierda  
        preorden(nodo.der, visitar) # 3º: derecha
```

8.2 Inorden: Izquierda → Raíz → Derecha

Resultado: **D, B, E, A, C, F**

En un **BST**, inorden produce los valores en **orden ascendente**. Extremadamente útil para obtener datos ordenados sin costo adicional.

Recorrido INORDEN (Izq → Raíz → Der)



Animación paso a paso del recorrido inorden.

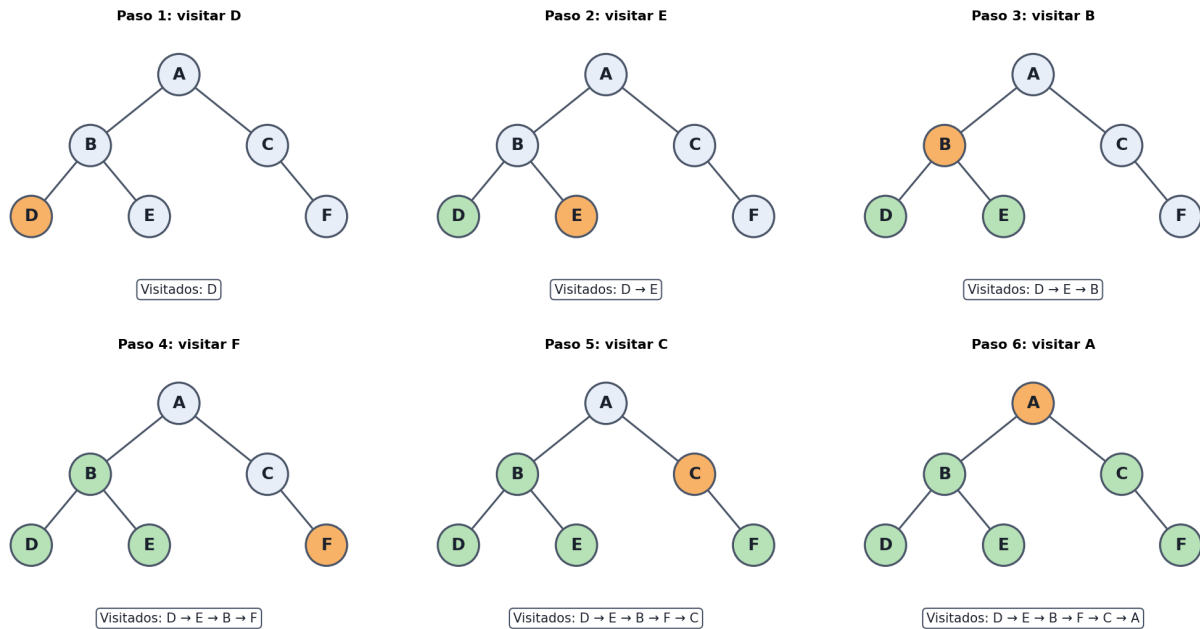
```
def inorden(nodo, visitar):
    if nodo:
        inorden(nodo.izq, visitar) # 1º: izquierda
        visitar(nodo.valor)        # 2º: raíz
        inorden(nodo.der, visitar) # 3º: derecha
```

8.3 Postorden: Izquierda → Derecha → Raíz

Resultado: **D, E, B, F, C, A**

Útil para **liberar memoria** (borrar hijos antes que padre) o **evaluar expresiones** (primero operandos, después el operador). En **backpropagation**, el flujo de gradientes en un grafo de cómputo es esencialmente un postorden.

Recorrido POSTORDEN (Izq → Der → Raíz)



Animación paso a paso del recorrido postorden.

```
def postorden(nodo, visitar):  
    if nodo:  
        postorden(nodo.izq, visitar) # 1º: izquierda  
        postorden(nodo.der, visitar) # 2º: derecha  
        visitar(nodo.valor)          # 3º: raíz
```

8.4 Recorridos iterativos y por niveles

Preorden iterativo con pila:

```
def preorden_iterativo(raiz):
    if not raiz: return
    pila = [raiz]
    while pila:
        nodo = pila.pop()
        print(nodo.valor)
        if nodo.der: pila.append(nodo.der)
        if nodo.izq: pila.append(nodo.izq)
```

Recorrido por niveles (BFS con cola):

```
from collections import deque
def por_niveles(raiz):
    if not raiz: return
    cola = deque([raiz])
    while cola:
        nodo = cola.popleft()
        print(nodo.valor)
        if nodo.izq: cola.append(nodo.izq)
        if nodo.der: cola.append(nodo.der)
```

Resultado por niveles: **A, B, C, D, E, F**

■ **Nota IA: Complejidad de todos los recorridos: $O(n)$** — visitamos cada nodo exactamente una vez.

9. Grafos y sus recorridos

Un **grafo** $G = (V, E)$ es un par formado por un conjunto de **vértices** V y un conjunto de **aristas** $E \subseteq V \times V$. Es la estructura más general y expresiva de la computación: cualquier árbol es un grafo, pero no al revés.

Tipos

- **Dirigido / No dirigido**: si las aristas tienen sentido.
- **Ponderado / No ponderado**: si las aristas tienen peso.
- **Cíclico / Acíclico**: un **DAG** es fundamental en IA.

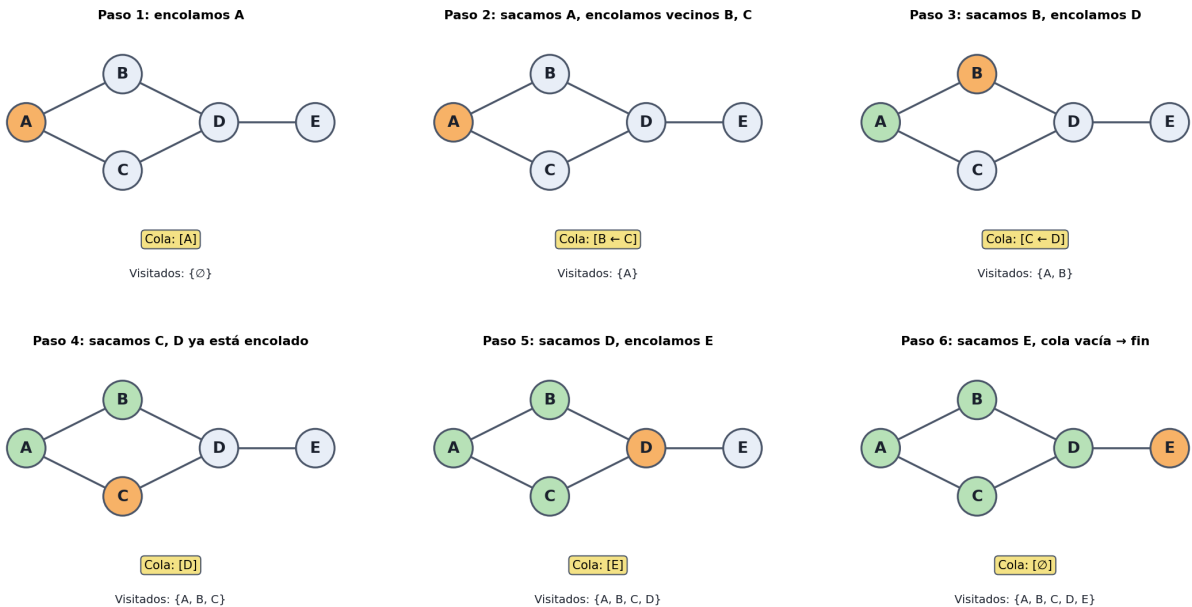
Los grafos en IA

- **Redes neuronales**: son DAGs de operaciones (PyTorch/TensorFlow).
- **Grafos de conocimiento**: Wikidata, DBpedia.
- **Redes sociales**: recomendaciones, detección de comunidades.
- **Graph Neural Networks (GNN)**: aprendizaje sobre grafos.
- **Redes bayesianas** y modelos probabilísticos.
- **Planificación**: estados como nodos, acciones como aristas.

9.1 BFS — Búsqueda en Anchura

Explora **por niveles**: primero todos los vecinos a distancia 1, luego a distancia 2, etc. Usa una **cola (FIFO)**. **Garantiza el camino más corto** en grafos no ponderados.

BFS - Búsqueda en Anchura desde A (usa COLA / FIFO)



Animación de BFS desde A. Naranja: nodo actual. Verde: visitado. Amarillo: en cola.

```
from collections import deque

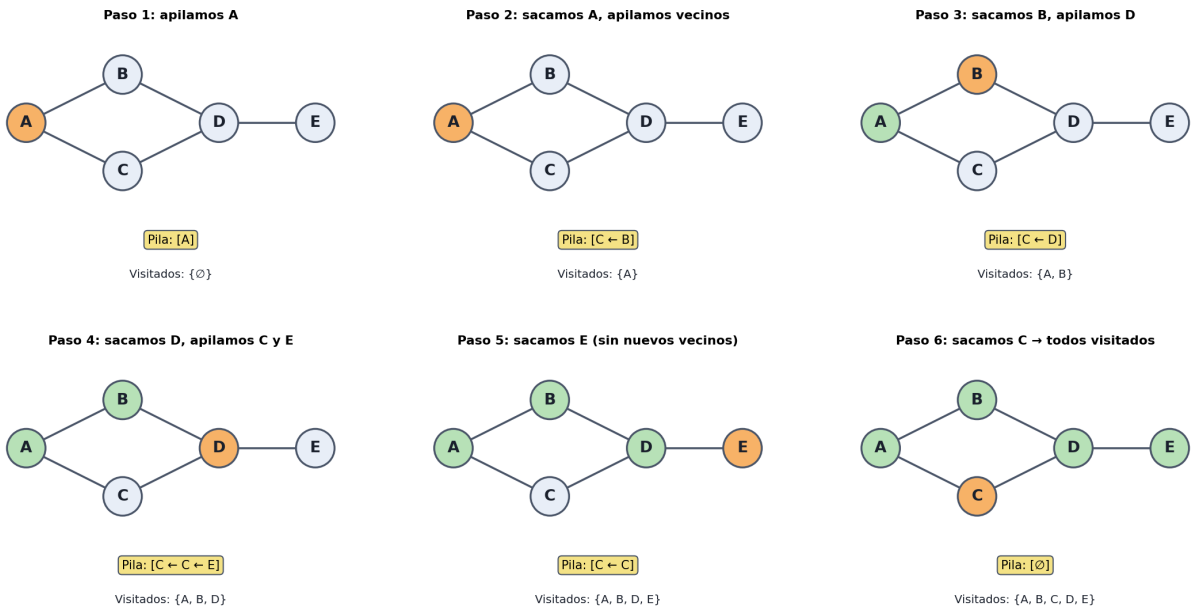
def bfs(grafo, inicio):
    visitados = {inicio}
    cola = deque([inicio])
    orden = []
    while cola:
        nodo = cola.popleft()
        orden.append(nodo)
        for vecino in grafo[nodo]:
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
    return orden
```

■ **Nota IA: Aplicaciones IA:** camino más corto en grafos no ponderados, niveles de conexión en redes sociales (grados de separación), difusión de información.

9.2 DFS — Búsqueda en Profundidad

Explora **lo más profundo posible antes de retroceder**. Usa una **pila** (o recursión, que usa la pila de llamadas del intérprete).

DFS - Búsqueda en Profundidad desde A (usa PILA / LIFO)



Animación de DFS iterativo desde A. Naranja: nodo actual. Verde: visitado. Amarillo: en pila.

```
def dfs(grafo, inicio, visitados=None):
    if visitados is None:
        visitados = set()
    visitados.add(inicio)
    print(inicio)
    for vecino in grafo[inicio]:
        if vecino not in visitados:
            dfs(grafo, vecino, visitados)

# Versión iterativa
def dfs_iterativo(grafo, inicio):
    visitados = set()
    pila = [inicio]
    while pila:
        nodo = pila.pop()
        if nodo not in visitados:
            visitados.add(nodo)
            for vecino in reversed(grafo[nodo]):
                if vecino not in visitados:
                    pila.append(vecino)
```

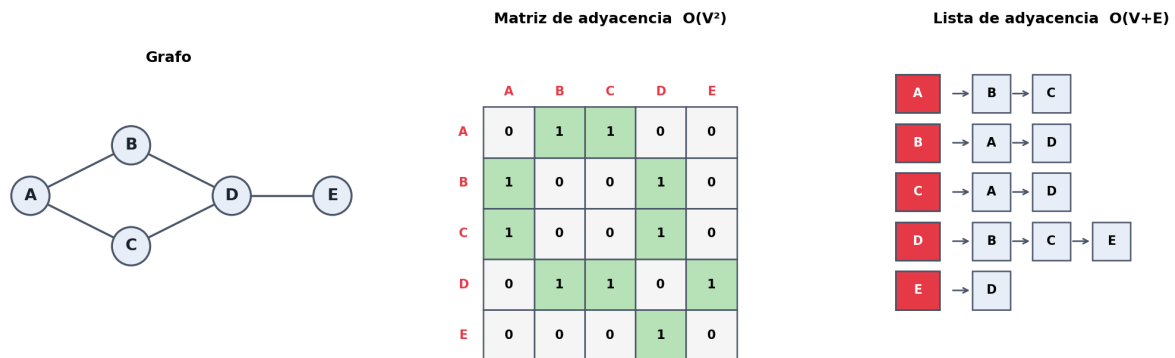
■ **Nota IA: Aplicaciones IA:** detección de ciclos, ordenamiento topológico (crucial para grafos de cómputo), componentes conexas, backtracking en CSP, resolución de laberintos.

BFS vs DFS

Aspecto	BFS	DFS
Estructura auxiliar	Cola (FIFO)	Pila (LIFO)
Camino más corto (no ponderado)	Sí	No garantiza
Memoria	$O(V)$ — nivel ancho	$O(h)$ — altura
Ideal para	Explorar cercanía	Explorar exhaustivamente
Compleitud	Sí (V finito)	Sí (V finito)
Complejidad	$O(V + E)$	$O(V + E)$

10. Lista y matriz de adyacencia

Son las dos formas canónicas de representar un grafo en memoria. La elección impacta radicalmente el rendimiento.



El mismo grafo representado como matriz (centro) y como lista de adyacencia (derecha).

10.1 Matriz de adyacencia

Matriz $V \times V$ donde $M[i][j] = 1$ (o el peso) si existe arista de i a j , y 0 en otro caso.

```
import numpy as np
M = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 0, 1, 0],
    [1, 0, 0, 1, 0],
    [0, 1, 1, 0, 1],
    [0, 0, 0, 1, 0]])
# ¿Existe arista A→B? M[0][1] == 1 → 0(1)
```

Pros: consulta de arista $O(1)$; álgebra lineal con NumPy; ideal para grafos **densos**.

Contras: memoria $O(V^2)$, prohibitivo para grafos grandes; iterar vecinos $O(V)$.

10.2 Lista de adyacencia

Para cada vértice, una lista (o conjunto) con sus vecinos. En Python se implementa habitualmente con un dict de listas.

```
grafo = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C', 'E'],
    'E': ['D']}
```

Pros: memoria $O(V+E)$, óptimo para grafos **dispersos**; iterar vecinos $O(\text{grado})$.

Contras: consulta de arista (u,v) es $O(\text{grado}(u))$; mejora a $O(1)$ usando set en vez de list.

Comparativa

Criterio	Matriz	Lista
Memoria	$O(V^2)$	$O(V + E)$
¿Existe arista (u,v) ?	$O(1)$	$O(\text{deg } u)$

Criterio	Matriz	Lista
Iterar vecinos de v	$O(V)$	$O(\deg v)$
Agregar arista	$O(1)$	$O(1)$
Ideal para	Densos	Dispersos
BFS/DFS	$O(V^2)$	$O(V + E)$

■ **Nota IA:** Un grafo con $V=10^8$ y $E=10^8$ ocupa 40 MB como lista (razonable) y 4 TB como matriz (imposible). En GNNs sobre grafos grandes (Facebook, Twitter) se usan formatos comprimidos como CSR o COO de `scipy.sparse` / `torch_geometric`. En Transformers, en cambio, las matrices de atención son densas y se usan matrices.

11. Complejidad temporal y espacial

Es la herramienta con la que **medimos y comparamos algoritmos** independientemente del hardware. Se expresa mediante **notación asintótica**.

Notaciones

- $O(f(n))$ — cota superior asintótica (peor caso).
- $\Omega(f(n))$ — cota inferior asintótica.
- $\Theta(f(n))$ — cota ajustada: crece exactamente como $f(n)$.

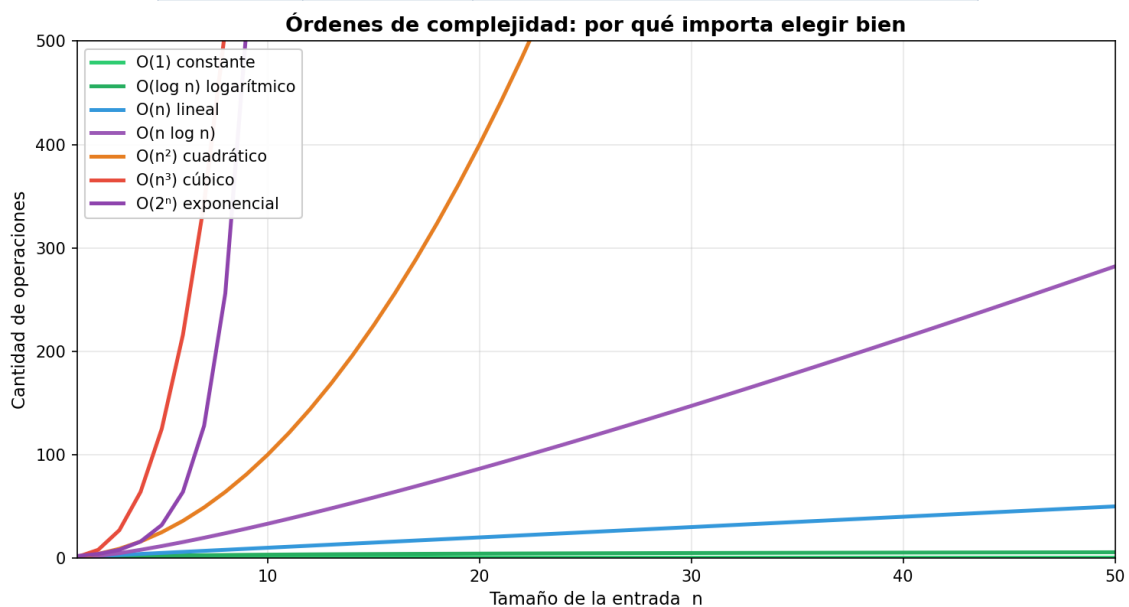
La más usada en la práctica es **O**.

Tipos

- **Temporal**: cantidad de operaciones básicas en función de n .
- **Espacial**: memoria adicional (más allá de la entrada).

Órdenes comunes

Orden	Nombre	Ejemplo típico
$O(1)$	Constante	Acceso a <code>list[i]</code> , <code>dict[k]</code>
$O(\log n)$	Logarítmico	Búsqueda binaria, operaciones en heap
$O(n)$	Lineal	Recorrer array, BFS/DFS sobre árbol
$O(n \log n)$	Casi lineal	Mergesort, Quicksort promedio
$O(n^2)$	Cuadrático	Bubble sort, matriz de distancias
$O(n^3)$	Cúbico	Multiplicación de matrices, Floyd-Warshall
$O(2^n)$	Exponencial	Subconjuntos, fuerza bruta en TSP
$O(n!)$	Factorial	Permutaciones, TSP exacto



Crecimiento de los distintos órdenes de complejidad.

Cómo se calcula

```
def ejemplo(A):
    n = len(A)
    for i in range(n):      # O(n)
        for j in range(n):  # O(n)
            print(A[i][j])  # O(1)
    for k in range(n):      # O(n)
        print(A[k][0])
# Total: O(n²) + O(n) = O(n²)
```

Complejidad amortizada

Analiza el **costo promedio por operación** a lo largo de una secuencia. Por ejemplo, `list.append` es $O(n)$ en el peor caso (cuando hay que redimensionar), pero **$O(1)$ amortizado** promediando sobre muchas inserciones.

Complejidades típicas en IA/ML

Algoritmo	Tiempo	Espacio
KNN (predicción)	$O(n \cdot d)$ por consulta	$O(n \cdot d)$
Regresión lineal (fórmula normal)	$O(n \cdot d^2 + d^3)$	$O(d^2)$
Árbol de decisión	$O(n \cdot d \cdot \log n)$	$O(\text{nodos})$
SVM (kernel)	$O(n^2 \text{ a } n^3)$	$O(n^2)$
Backprop (una época)	$O(W)$ W = parámetros	$O(W)$
Self-attention (Transformer)	$O(L^2 \cdot d)$	$O(L^2)$
A* (peor caso)	$O(b^{\frac{1}{2}})$	$O(b^{\frac{1}{2}})$

■ **Nota IA:** El $O(L^2)$ del Transformer respecto a la longitud de secuencia L es el motivo por el cual procesar documentos largos con LLMs es costoso y por qué existen variantes como **Longformer**, **Reformer** y **FlashAttention**.

Ejemplo comparativo: búsqueda

```
# Búsqueda lineal → O(n)
def buscar_lineal(A, x):
    for i, v in enumerate(A):
        if v == x: return i
    return -1

# Búsqueda binaria (A ordenado) → O(log n)
def buscar_binaria(A, x):
    lo, hi = 0, len(A) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if A[mid] == x: return mid
        elif A[mid] < x: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Para $n = 10^6$: la lineal hace hasta 10^6 comparaciones, la binaria unas 30. Esa es la diferencia entre *instantáneo* y *no termina nunca*.

Cierre

Retengan estas tres ideas rectoras:

- **La estructura de datos correcta cambia el orden de magnitud del algoritmo.** Un set en lugar de una list para pertenencia puede convertir un $O(n^2)$ en $O(n)$.
- **Los TDAs separan el qué del cómo.** Es lo que permite que un mismo algoritmo funcione con distintas implementaciones subyacentes.
- **En IA, los grafos y árboles son ubicuos.** Redes neuronales, GNNs, árboles de decisión, MCTS, búsqueda heurística: todo se apoya en estas estructuras.

Recomendación: implementen cada estructura desde cero al menos una vez, aunque en el trabajo real usen `heapq`, `collections`, `networkx` o `torch_geometric`. Entender el mecanismo interno les dará la intuición para diagnosticar problemas de rendimiento en modelos reales.